

Balanceo de carga en clústeres de servidores mediante factorización LU

Solución de sistemas de ecuaciones lineales por el método de Doolittle

Estudiante: **Fernando Istaña Flores** Docente: **Jorge Luis Manrique Plasencia**

Escuela: **Ingeniería de Sistemas**

1. Planteamiento del problema

Una empresa de infraestructura Cloud administra un clúster de microservicios distribuidos cuyo rendimiento global depende de las latencias de procesamiento de cada nodo. Las dependencias entre servicios se modelan con el sistema lineal $A \cdot x = b$, donde A reúne los coeficientes de transferencia y acoplamiento entre servicios, b contiene las latencias máximas permitidas por el SLA y x los tiempos de cómputo que se busca determinar.

Dado que el vector de peticiones b cambia continuamente con el tráfico web, recalculiar la eliminación gaussiana completa en cada petición resulta ineficiente. La estrategia adecuada consiste en descomponer la matriz A una sola vez como $A = LU$ y resolver cada nuevo vector mediante dos sustituciones de complejidad $O(n^2)$.

Sistema del clúster (3 microservicios)

$$\begin{aligned} 4x_1 + 2x_2 + 1x_3 &= 14 && (\text{tráfico global}) \\ 12x_1 + 10x_2 + 5x_3 &= 46 && (\text{procesamiento backend}) \\ -8x_1 + 8x_2 + 7x_3 &= 26 && (\text{consultas a base de datos}) \end{aligned}$$

Vectores de tráfico evaluados: $b_1 = (14, 46, 26)^T$, correspondiente al SLA inicial, y $b_2 = (20, 62, 30)^T$, correspondiente a un pico de tráfico en tiempo real.

2. Fase 1 — Descomposición $A = LU$ por el método de Doolittle

El método de Doolittle fija la diagonal de L en 1 y almacena en las posiciones inferiores los multiplicadores usados en la eliminación; la matriz U es el resultado triangular superior de esa misma eliminación.

- Columna 1** (pivote $u_{11} = 4$): $m_{21} = 12/4 = 3$ y $m_{31} = -8/4 = -2$. Se aplican $F_2 \leftarrow F_2 - 3 \cdot F_1$ y $F_3 \leftarrow F_3 + 2 \cdot F_1$, obteniendo las filas $(0, 4, 2)$ y $(0, 12, 9)$.
- Columna 2** (pivote $u_{22} = 4$): $m_{32} = 12/4 = 3$. Se aplica $F_3 \leftarrow F_3 - 3 \cdot F_2$, obteniendo la fila $(0, 0, 3)$.
- Columna 3**: el pivote $u_{33} = 3$ cierra la eliminación; no quedan filas debajo.

L	U	L · U = A
1 0 0	4 2 1	4 2 1
3 1 0	0 4 2	12 10 5
-2 3 1	0 0 3	-8 8 7

El producto $L \cdot U$ reproduce exactamente la matriz original, lo que confirma la factorización. Además, $\det(A) = u_{11} \cdot u_{22} \cdot u_{33} = 4 \cdot 4 \cdot 3 = \mathbf{48 \neq 0}$, de modo que el sistema es compatible determinado y admite una única solución.

3. Fase 1 — Solución para el vector b_1

Sustitución progresiva: $L \cdot y = b_1$

$$y_1 = 14$$

$$y_2 = 46 - 3(14) = 4$$

$$y_3 = 26 - (-2)(14) - 3(4) = 26 + 28 - 12 = 42$$

Sustitución regresiva: $U \cdot x = y$

$$x_3 = 42 / 3 = 14$$

$$x_2 = (4 - 2(14)) / 4 = -24 / 4 = -6$$

$$x_1 = (14 - 2(-6) - 1(14)) / 4 = 12 / 4 = 3$$

Resultado: $x = (3, -6, 14)^T$. La verificación directa confirma las tres ecuaciones: $4(3) + 2(-6) + 14 = 14$; $12(3) + 10(-6) + 5(14) = 46$; $-8(3) + 8(-6) + 7(14) = 26$.

4. Fase 2 — Implementación del algoritmo

El programa se escribió en Python sin bibliotecas externas, con la factorización separada de la resolución. Esto permite llamar a `doolittle(A)` una única vez y reutilizar L y U en cada nuevo vector de tráfico, que es precisamente la optimización que exige el enunciado.

```
def doolittle(A):
    """A = L U con L triangular inferior de diagonal unitaria."""
    n = len(A)
    L = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    U = [row[:] for row in A]
    for k in range(n):
        if abs(U[k][k]) < 1e-12:
            raise ValueError("pivote nulo: se requiere pivoteo parcial")
        for i in range(k + 1, n):
            m = U[i][k] / U[k][k]          # multiplicador -> entrada de L
            L[i][k] = m
            for j in range(k, n):
                U[i][j] -= m * U[k][j]
            U[i][k] = 0.0
    return L, U

def resolver(L, U, b):
```

```

n = len(b)
y = [0.0] * n # L y = b (sustitucion progresiva)
for i in range(n):
    y[i] = (b[i] - sum(L[i][j] * y[j] for j in range(i))) / L[i][i]
x = [0.0] * n # U x = y (sustitucion regresiva)
for i in range(n - 1, -1, -1):
    x[i] = (y[i] - sum(U[i][j] * x[j] for j in range(i + 1, n))) / U[i][i]
return y, x

A = [[4, 2, 1], [12, 10, 5], [-8, 8, 7]]
b1 = [14, 46, 26] # SLA inicial
b2 = [20, 62, 30] # pico de trafico

L, U = doolittle(A) # UNA sola vez, costo 0(n^3)
for nombre, b in [("b1", b1), ("b2", b2)]:
    y, x = resolver(L, U, b) # solo 0(n^2) por vector
    r = [sum(A[i][j] * x[j] for j in range(3)) - b[i] for i in range(3)]
    print(nombre, "y =", y, "x =", x, "residuo =", r)

```

Salida del programa

```

=====
FASE 1 – Descomposicion A = LU por Doolittle (una sola vez)
=====
L =
[ 1.0000  0.0000  0.0000]
[ 3.0000  1.0000  0.0000]
[ -2.0000  3.0000  1.0000]
U =
[ 4.0000  2.0000  1.0000]
[ 0.0000  4.0000  2.0000]
[ 0.0000  0.0000  3.0000]
det(A) = 48.0000

=====
FASE 1 y 2 – Sustituciones para cada vector de trafico
=====
b1 = (14.0000, 46.0000, 26.0000)
y = (14.0000, 4.0000, 42.0000) (sustitucion progresiva)
x = (3.0000, -6.0000, 14.0000) (sustitucion regresiva)
residuo A.x - b = (0.0000, 0.0000, 0.0000)

b2 = (20.0000, 62.0000, 30.0000)
y = (20.0000, 2.0000, 64.0000) (sustitucion progresiva)
x = (4.7500, -10.1667, 21.3333) (sustitucion regresiva)
residuo A.x - b = (0.0000, 0.0000, -0.0000)

Costo con LU : 18 + 2x18 = 54 flops
Costo recalculando : 2x(18 + 18) = 72 flops

```

5. Fase 2 — Cambio de tráfico en tiempo real (vector b₂)

Al llegar el segundo vector de peticiones no se repite la descomposición: las matrices L y U ya calculadas entran directamente a las dos sustituciones.

$$\begin{aligned}
 y_1 &= 20 & y_2 &= 62 - 3(20) = 2 & y_3 &= 30 + 2(20) - 3(2) = 64 \\
 x_3 &= 64/3 \approx 21.3333 & x_2 &= (2 - 2(64/3))/4 \approx -10.1667 & x_1 &= 19/4 = 4.75
 \end{aligned}$$

VECTOR DE TRÁFICO	B	Y (PROGRESIVA)	X (SOLUCIÓN)	$\ A \cdot X - B\ _\infty$
b_1 – SLA inicial	(14, 46, 26)	(14, 4, 42)	(3, -6, 14)	0
b_2 – pico de tráfico	(20, 62, 30)	(20, 2, 64)	(4.7500, -10.1667, 21.3333)	≈ 0

6. Análisis de resultados

Verificación numérica. En ambos casos el residuo $A \cdot x - b$ es nulo hasta la precisión de máquina, de modo que las soluciones son exactas y la factorización se realizó correctamente.

Costo computacional. Para $n = 3$, la factorización requiere aproximadamente $2n^3/3 = 18$ operaciones en punto flotante y cada par de sustituciones $2n^2 = 18$. Resolver los dos vectores con LU cuesta $18 + 2(18) = 54$ flops, frente a los 72 flops que exigiría repetir la eliminación gaussiana completa en cada petición. La ventaja crece con el número de vectores: para q oleadas el costo es $2n^3/3 + q \cdot 2n^2$ en lugar de $q(2n^3/3 + 2n^2)$, y también crece con el tamaño del clúster, pues el término cúbico deja de repetirse.

Interpretación en el clúster. Los tiempos obtenidos para x_2 resultan negativos en ambos escenarios, lo que carece de sentido físico como latencia de procesamiento. Esto indica que el modelo de acoplamiento exige al microservicio S_2 liberar carga en lugar de consumirla para cumplir el SLA propuesto, es decir, que las restricciones de latencia son inconsistentes con la arquitectura actual. En términos operativos, el resultado sugiere revisar los coeficientes de transferencia de ese nodo o renegociar el SLA antes de aplicar el balanceo.

7. Conclusiones

La factorización LU por el método de Doolittle resolvió el sistema del clúster con una única descomposición de la matriz de acoplamiento, obteniendo $x = (3, -6, 14)^T$ para el SLA inicial y $x \approx (4.75, -10.1667, 21.3333)^T$ para el pico de tráfico. La separación entre factorización y sustitución es lo que hace viable el recálculo en tiempo real: el trabajo cúbico se paga una sola vez y cada nueva petición se atiende con dos sustituciones de orden cuadrático. Finalmente, el signo de las soluciones muestra que el análisis numérico no termina en el cálculo: la interpretación del resultado es la que permite detectar que el modelo de dependencias del clúster necesita ajuste.